

Hadoop programs

Program 2: WordCount

```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    // MAPPER
    public static class MyMapper
        extends Mapper<Object, Text, Text, IntWritable> {

        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

            String[] words = value.toString().split(" ");

            for (String word : words) {
                context.write(
                    new Text(word),
                    new IntWritable(1)
                );
            }
        }
    }

    // REDUCER
    public static class MyReducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {

        public void reduce(Text key, Iterable<IntWritable> values,
            Context context)
            throws IOException, InterruptedException {

            int sum = 0;
```

```

        for (IntWritable val : values) {
            sum += val.get();
        }

        context.write(key, new IntWritable(sum));
    }
}

// DRIVER
public static void main(String[] args) throws Exception {

    Configuration conf = new Configuration();

    Job job = Job.getInstance(conf, "wordcount");

    job.setJarByClass(WordCount.class);

    job.setMapperClass(MyMapper.class);
    job.setReducerClass(MyReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

Program 3: Max average student marks

```

import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MaximumAverage {

```

```
// ===== MAPPER =====
public static class MyMapper
    extends Mapper<Object, Text,
        Text, FloatWritable> {

    public void map(Object key,
        Text value,
        Context context)
        throws IOException, InterruptedException {

        String line = value.toString();

        String[] marks = line.split(" ");

        int sum = 0;

        for (String mark : marks) {
            sum += Integer.parseInt(mark);
        }

        float avg = (float) sum / marks.length;

        context.write(
            new Text("maxavg"),
            new FloatWritable(avg)
        );
    }
}

// ===== REDUCER =====
public static class MyReducer
    extends Reducer<Text, FloatWritable,
        Text, FloatWritable> {

    public void reduce(Text key,
        Iterable<FloatWritable> values,
        Context context)
        throws IOException, InterruptedException {

        float max = 0;

        for (FloatWritable val : values) {

            if (val.get() > max) {
                max = val.get();
            }
        }

        context.write(
```

```

        key,
        new FloatWritable(max)
    );
}
}

// ===== DRIVER =====
public static void main(String[] args)
    throws Exception {

    Configuration conf = new Configuration();

    Job job = Job.getInstance(conf, "maxaverage");

    job.setJarByClass(MaximumAverage.class);

    job.setMapperClass(MyMapper.class);
    job.setReducerClass(MyReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(FloatWritable.class);

    FileInputFormat.addInputPath(
        job,
        new Path(args[0])
    );

    FileOutputFormat.setOutputPath(
        job,
        new Path(args[1])
    );

    System.exit(
        job.waitForCompletion(true) ? 0 : 1
    );
}
}

```

Program 4: Max temp

```

import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;

```

```

import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MaxTemperature {

    // ===== MAPPER =====
    public static class MyMapper
        extends Mapper<Object, Text,
            Text, FloatWritable> {

        public void map(Object key,
            Text value,
            Context context)
            throws IOException, InterruptedException {

            String line = value.toString();

            // avoid empty lines
            if (line.length() > 0) {

                // extract year
                String year = line.substring(6, 10);

                // extract temperature
                float temp = Float.parseFloat(
                    line.substring(54, 62).trim()
                );

                context.write(
                    new Text(year),
                    new FloatWritable(temp)
                );
            }
        }
    }

    // ===== REDUCER =====
    public static class MyReducer
        extends Reducer<Text, FloatWritable,
            Text, FloatWritable> {

        public void reduce(Text key,
            Iterable<FloatWritable> values,
            Context context)
            throws IOException, InterruptedException {

            float max = Float.MIN_VALUE;

```

```

        for (FloatWritable val : values) {

            if (val.get() > max) {
                max = val.get();
            }
        }

        context.write(
            key,
            new FloatWritable(max)
        );
    }
}

// ===== DRIVER =====
public static void main(String[] args)
    throws Exception {

    Configuration conf = new Configuration();

    Job job = Job.getInstance(conf, "maxtemp");

    job.setJarByClass(MaxTemperature.class);

    job.setMapperClass(MyMapper.class);
    job.setReducerClass(MyReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(FloatWritable.class);

    FileInputFormat.addInputPath(
        job,
        new Path(args[0])
    );

    FileOutputFormat.setOutputPath(
        job,
        new Path(args[1])
    );

    System.exit(
        job.waitForCompletion(true) ? 0 : 1
    );
}
}

```

Program 6: Diff of fuel consumption of electric and petrol cars

```

import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class PetroleumDifference {

    // ===== MAPPER =====
    public static class MyMapper
        extends Mapper<Object, Text,
            Text, FloatWritable> {

        public void map(Object key,
            Text value,
            Context context)
            throws IOException, InterruptedException {

            String line = value.toString();

            String[] data = line.split(",");

            // avoid invalid rows
            if (data.length > 49) {

                String vehicleType = data[49];
                String fuel = data[0];

                // check fuel is numeric
                if (fuel.matches("-?\\d+(\\.\\d+)?")) {

                    context.write(
                        new Text(vehicleType),
                        new FloatWritable(
                            Float.parseFloat(fuel)
                        )
                    );
                }
            }
        }
    }
}

```

```

    }
}

// ===== REDUCER =====
public static class MyReducer
    extends Reducer<Text, FloatWritable,
        Text, FloatWritable> {

    float electricAvg = 0;
    float gasolineAvg = 0;

    public void reduce(Text key,
        Iterable<FloatWritable> values,
        Context context)
        throws IOException, InterruptedException {

        float sum = 0;
        float count = 0;

        for (FloatWritable val : values) {
            sum += val.get();
            count++;
        }

        float avg = sum / count;

        // store averages
        if (key.toString().equals("true")) {
            electricAvg = avg;
        }
        else if (key.toString().equals("false")) {
            gasolineAvg = avg;
        }
    }

    // runs after all reduce calls
    protected void cleanup(Context context)
        throws IOException, InterruptedException {

        float difference =
            Math.abs(
                electricAvg - gasolineAvg
            );

        context.write(
            new Text(
                "Difference of petroleum consumption"
            ),
            new FloatWritable(difference)
        );
    }
}

```



```

    }
}

// ===== DRIVER =====
public static void main(String[] args)
    throws Exception {

    Configuration conf = new Configuration();

    Job job = Job.getInstance(
        conf,
        "petroleum difference"
    );

    job.setJarByClass(
        PetroleumDifference.class
    );

    job.setMapperClass(MyMapper.class);
    job.setReducerClass(MyReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(FloatWritable.class);

    FileInputFormat.addInputPath(
        job,
        new Path(args[0])
    );

    FileOutputFormat.setOutputPath(
        job,
        new Path(args[1])
    );

    System.exit(
        job.waitForCompletion(true) ? 0 : 1
    );
}
}

```

Program 7: average fuel consumption of electric cars

```

import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.Text;

```

```

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class AverageCityMPG {

    // ===== MAPPER =====
    public static class MyMapper
        extends Mapper<Object, Text,
            Text, FloatWritable> {

        public void map(Object key,
            Text value,
            Context context)
            throws IOException, InterruptedException {

            String line = value.toString();

            String[] data = line.split(",");

            // avoid invalid rows
            if (data.length > 49) {

                String vehicleType = data[49];
                String cityMPG = data[4];

                // check MPG is numeric
                if (cityMPG.matches("-?\\d+(\\.\\d+)?")) {

                    context.write(
                        new Text(vehicleType),
                        new FloatWritable(
                            Float.parseFloat(cityMPG)
                        )
                    );
                }
            }
        }

        // ===== REDUCER =====
        public static class MyReducer
            extends Reducer<Text, FloatWritable,
                Text, FloatWritable> {

            public void reduce(Text key,

```

```

        Iterable<FloatWritable> values,
        Context context)
        throws IOException, InterruptedException {

    // only electric cars
    if (key.toString().equals("true")) {

        float sum = 0;
        float count = 0;

        for (FloatWritable val : values) {

            sum += val.get();
            count++;
        }

        float avg = sum / count;

        context.write(
            new Text(
                "Average city MPG for electric cars"
            ),
            new FloatWritable(avg)
        );
    }
}

// ===== DRIVER =====
public static void main(String[] args)
    throws Exception {

    Configuration conf = new Configuration();

    Job job = Job.getInstance(
        conf,
        "average city mpg"
    );

    job.setJarByClass(
        AverageCityMPG.class
    );

    job.setMapperClass(MyMapper.class);
    job.setReducerClass(MyReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(FloatWritable.class);

    FileInputFormat.addInputPath(

```

```

        job,
        new Path(args[0])
    );

    FileOutputFormat.setOutputPath(
        job,
        new Path(args[1])
    );

    System.exit(
        job.waitForCompletion(true)
        ? 0 : 1
    );
}
}

```

Program 8: find even / odd numbers

```

import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class EvenOdd {

    // ===== MAPPER =====
    public static class MyMapper
        extends Mapper<Object, Text,
            Text, Text> {

        public void map(Object key,
            Text value,
            Context context)
            throws IOException, InterruptedException {

            String number = value.toString();

            int num = Integer.parseInt(number);

```

```

        if (num % 2 == 0) {

            context.write(
                new Text(number),
                new Text("Even")
            );
        }
        else {

            context.write(
                new Text(number),
                new Text("Odd")
            );
        }
    }
}

// ===== REDUCER =====
public static class MyReducer
    extends Reducer<Text, Text,
        Text, Text> {

    public void reduce(Text key,
        Iterable<Text> values,
        Context context)
        throws IOException, InterruptedException {

        for (Text val : values) {

            context.write(key, val);
        }
    }
}

// ===== DRIVER =====
public static void main(String[] args)
    throws Exception {

    Configuration conf = new Configuration();

    Job job = Job.getInstance(
        conf,
        "evenodd"
    );

    job.setJarByClass(EvenOdd.class);

    job.setMapperClass(MyMapper.class);
    job.setReducerClass(MyReducer.class);

```

```

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        FileInputFormat.addInputPath(
            job,
            new Path(args[0])
        );

        FileOutputFormat.setOutputPath(
            job,
            new Path(args[1])
        );

        System.exit(
            job.waitForCompletion(true)
                ? 0 : 1
        );
    }
}

```

Program 9: find prime numbers

```

import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class PrimeNumbers {

    // ===== MAPPER =====
    public static class MyMapper
        extends Mapper<Object, Text,
            Text, Text> {

        public void map(Object key,
            Text value,
            Context context)
            throws IOException, InterruptedException {

            String number = value.toString();

```

```

        context.write(
            new Text(number),
            new Text("")
        );
    }
}

// ===== REDUCER =====
public static class MyReducer
    extends Reducer<Text, Text,
        Text, Text> {

    public void reduce(Text key,
        Iterable<Text> values,
        Context context)
        throws IOException, InterruptedException {

        int num =
            Integer.parseInt(key.toString());

        boolean isPrime = true;

        // 0 and 1 are not prime
        if (num <= 1) {
            isPrime = false;
        }

        for (int i = 2;
            i <= num / 2;
            i++) {

            if (num % i == 0) {
                isPrime = false;
                break;
            }
        }

        if (isPrime) {

            context.write(
                key,
                new Text("Prime")
            );
        }
    }
}

// ===== DRIVER =====
public static void main(String[] args)

```

```

        throws Exception {

    Configuration conf = new Configuration();

    Job job = Job.getInstance(
        conf,
        "prime numbers"
    );

    job.setJarByClass(
        PrimeNumbers.class
    );

    job.setMapperClass(MyMapper.class);
    job.setReducerClass(MyReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);

    FileInputFormat.addInputPath(
        job,
        new Path(args[0])
    );

    FileOutputFormat.setOutputPath(
        job,
        new Path(args[1])
    );

    System.exit(
        job.waitForCompletion(true)
        ? 0 : 1
    );
}
}

```

Program 10: total and average salaries

```

import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;

```



```

import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class SalaryAverageTotal {

    // ===== MAPPER =====
    public static class MyMapper
        extends Mapper<Object, Text,
            Text, FloatWritable> {

        public void map(Object key,
            Text value,
            Context context)
            throws IOException, InterruptedException {

            String line = value.toString();

            String[] data = line.split("\\t");

            // gender
            String gender = data[3];

            // salary
            float salary =
                Float.parseFloat(data[8]);

            context.write(
                new Text(gender),
                new FloatWritable(salary)
            );
        }
    }

    // ===== REDUCER =====
    public static class MyReducer
        extends Reducer<Text, FloatWritable,
            Text, Text> {

        public void reduce(Text key,
            Iterable<FloatWritable> values,
            Context context)
            throws IOException, InterruptedException {

            float total = 0;
            int count = 0;

            for (FloatWritable val : values) {

```

```

        total += val.get();
        count++;
    }

    float average = total / count;

    String result =
        "Total = " + total +
        " Average = " + average;

    context.write(
        key,
        new Text(result)
    );
}
}

// ===== DRIVER =====
public static void main(String[] args)
    throws Exception {

    Configuration conf = new Configuration();

    Job job = Job.getInstance(
        conf,
        "salary average total"
    );

    job.setJarByClass(
        SalaryAverageTotal.class
    );

    job.setMapperClass(MyMapper.class);
    job.setReducerClass(MyReducer.class);

    job.setMapOutputKeyClass(Text.class);
    job.setMapOutputValueClass(FloatWritable.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);

    FileInputFormat.addInputPath(
        job,
        new Path(args[0])
    );

    FileOutputFormat.setOutputPath(
        job,
        new Path(args[1])
    );
}

```

```
        System.exit(  
            job.waitForCompletion(true)  
            ? 0 : 1  
        );  
    }  
}
```